

MSU Graduate Spatial Ecology Lab 2

Phoebe Zarnetske: plz@msu.edu; Lala Kounta: kountala@msu.edu; Kelly Kapsar: kapsarke@msu.edu

Last rev. Sep, 2025; created Sep, 2020

Lab 2: R Markdown, Intro to mapping in R & Scale

[RMarkdown version of this Lab](#)

This lab has 2 Parts. You need to hand in Part 2 via the D2L Lab 2 Assignment Folder by start of Lab 3. When referring to the code below it may be more useful to use the .Rmd file linked above. You will need to be connected to the internet to complete this lab.

At the end of Lab 2 is Homework in preparation for Lab 3.

Part 1: R Markdown

R Markdown is an authoring framework for R code and output. R Markdown is part word-processor, part R output. It enables you to generate nice reports with R code, figures, tables, and text. It's handy because it produces neat summaries of your work in HTML or PDF or other formats (Word docs). This document was made in Markdown via RStudio. From now on, you'll be handing in all of your lab assignments as PDFs produced from R Markdown.

R Markdown is especially helpful with collaborative research and coursework, and is often used in supplemental materials with publications. If you haven't already read the [R Markdown tutorial](#), you may want to refer to it when putting together your lab write-up (including "How it Works", "Code Chunks", and "Markdown Basics"). I recommend using "KnitR" in RStudio when you want to publish (i.e., click on "knit to HTML", "knit to PDF", etc. in the pull-down menu in RStudio).

When you click the **Knit** button in R Studio (looks like a knitting needle with ball of yarn) a document will be generated that includes both content as well as the output of any embedded R code chunks within the document.

Help with knitting to PDF

You may need to install a TEX program to knit to PDF. Try knitting first to PDF, if this doesn't work here is a suggestion (this may take a while to download):

For Mac: [MacTex](#) For PC: [MikTex](#)

If you have trouble knitting to PDF directly in RStudio, you can also save to HTML and when you open the HTML in a browser, you can "save as PDF" or "print to PDF" directly from the browser. Just make sure what you end up handing in originated as a .Rmd file and presents the code and plots and lab write-up clearly.

Interested in more detail on how to use RMarkdown and knitr?

Here is a great [mini tutorial from EarthLab](#) illustrating how to get started with RMarkdown in RStudio. Note that Lesson 2 is the intro to RMarkdown & Knitr.

[EarthLab's Lesson 5](#) goes over how to knit to PDF.

Some basics of RMarkdown are included below:

In RMarkdown, you can embed an R code chunk like this:

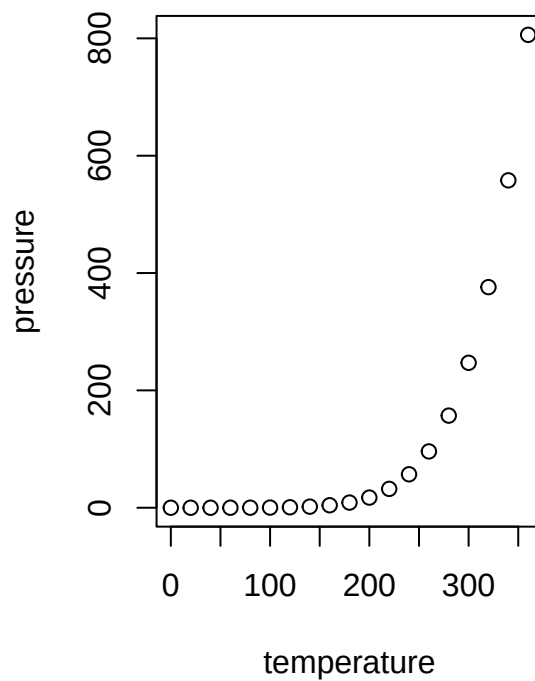
```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
## 1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##   Mean  :15.4    Mean   : 42.98
## 3rd Qu.:19.0    3rd Qu.: 56.00
##   Max.  :25.0    Max.    :120.00
```

```
summary(cars)
```

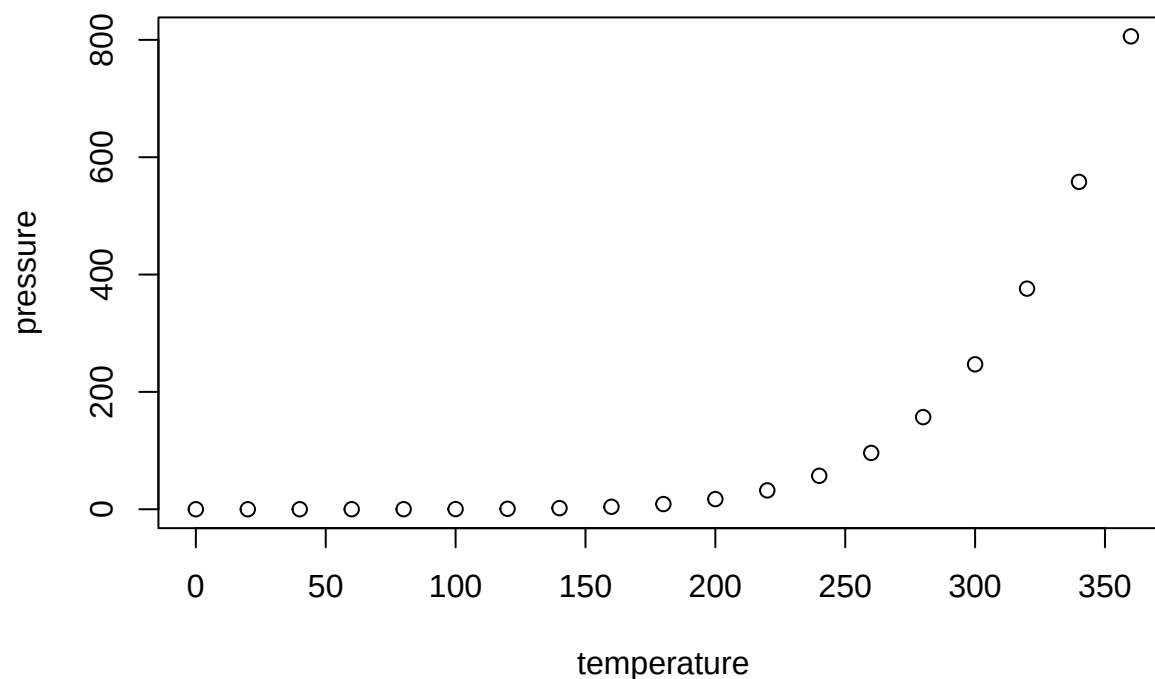
`results = 'hide'` will omit the printout (suppressing the summary of the dataframe cars) R Markdown can print or hide certain portions of your code and output. This is helpful especially when you don't want to include long print-outs in a report. For lab assignments, please don't include long R printouts, e.g., printing an entire dataset.

Including Plots



You can also embed plots, for example with the dataframe, pressure:

`echo = FALSE` will not print the code beneath it; `TRUE` is the default. You can also set a figure height and width in the brackets also. Units = inches.



`warning = FALSE` will not print the R warnings beneath it; `TRUE` is the default.

Part 2: Intro to mapping in R & Scale

R is ok for making maps - it's getting better. If you want something super pretty or very complicated, it is probably better to go with QGIS or ArcGIS. However, you can make some nice maps in R with some basic commands, and because it's all scripted, it's useful for reproducible research.

```
##### STARTING UP R
# Clear all existing data
rm(list=ls())

# Close graphics devices
graphics.off()

# Set the paths for your work
output_path<-"output"
# if this folder doesn't exist, create it
if(!dir.exists(output_path)){
  dir.create(output_path)
}

# Create the folders (directories) "data" and "lab2" - If they exist already, this
# command won't over-write them.
```

```

data_path<-(file.path("data","lab2"))
if(!dir.exists(data_path)){
  dir.create(data_path,recursive = TRUE)
}

# If you previously saved your workspace and want to load it here, do so this way
# load(file.path(output_path,"lab2.RData"))

# With R Markdown, it is helpful to install packages locally before knitting
# (copy this into your R console before you knit)
for (package in c("raster","sf","terra","dismo")) {
  if (!require(package, character.only=T, quietly=T)) {
    install.packages(package)
    library(package, character.only=T)
  }
}

# If you get a warning that says: there is no package called 'raster'
# (or another package) type into your R console:
# package(raster)
# library(raster)

```

Coordinate Reference Systems & Projection

Check out [this helpful \(and fun\) video](#) that explains projections and CRS. To see why every map we view is relative, check out [this interactive site](#).

Download and read in some spatial data. Spatial data require spatial reference. A common way of defining the spatial reference is with:

proj4 = projection information about the spatial data, and

CRS = coordinate reference systems containing projection, datum, and ellipsoid.

A CRS provides the key for how to interpret spatially referenced data given a specific geometry. It provides information about how to transform a three dimensional angular system into a 2-dimensional planar system. [Great overview](#) for the proj4 strings see www.spatialreference.org or search on <https://epsg.io/>

```

# USA BOUNDARY SHAPEFILE: download it from the web [note that you may need to
# change the exdir location depending on where you want it to be saved]
if(! file.exists(file.path(data_path,'us.zip'))){
  download.file("http://www2.census.gov/geo/tiger/GENZ2014/shp/cb_2014_us_nation_20m.zip",
    dest=file.path(data_path,"us.zip"), mode="wb")
}

unzip (file.path(data_path,"us.zip"), exdir = data_path)
# Read in the shapefile with rgdal package
us <- sf::read_sf(file.path(data_path,"cb_2014_us_nation_20m.shp"))
# Note that shapefiles are actually a collection of many files. If you move a
# shapefile manually, be sure to copy all of the files.

# take a look at the shapefile info
summary(us)

```

Uh oh! It's not projected, is that a problem? The proj4string states that it's a geographic coordinate system (rather than a projected coordinate system). Geographic systems include angular unit of measure, a prime meridian, and a datum (based on a spheroid). Latitude and Longitude describe any location in geographic coordinate system. If we wanted to do some exact measurements of distances or areas, it would be better to use a projection. This is because projections transform a 3D surface (Earth; a sphere) into a 2D surface and maintain constant lengths and angles, preserving area. Geographic systems have different lengths and angles depending on the location within the spatial data. Geographic systems are often used with Google Maps.

Projecting Data

```
## This defines the CRS for the US National Atlas Map Projection
us.atlas.proj <- "EPSG:2163"
# set a plotting area for 2 side-by-side subplots
par(mfcol=c(1,2))
# plot the shapefile in the first subplot
plot(us$geometry)
# project to US National Atlas equal-area projection
us.equal <- sf::st_transform(us, us.atlas.proj)
# plot the other shapefile in the second subplot
plot(us.equal$geometry)
```



Reading in data from gbif.org - the Global Biodiversity Information Facility

Add some point data for a species from GBIF.org. Get GBIF data with function `gbif()` from the *dismo* R package or choose your own favorite organism (in the US). Check the GBIF site first to see how many occurrences you'll be importing <http://www.gbif.org/occurrence/search> - enter your species name in the search window. Click on "filter" and scientific name. Puma concolor has 26,600+ records, and may take a few mins to import. Choose a different species in the US with a smaller number if you don't want to wait so long.

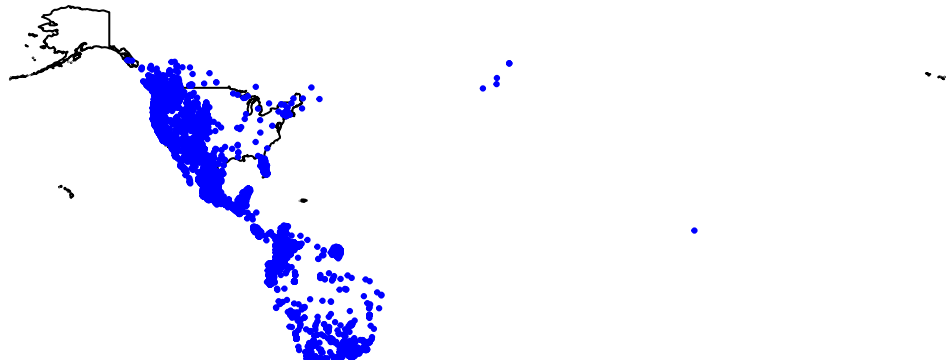
It might take time to import the occurrence data on puma through GBIF. Alternatively, and for the purpose of this lab only, you can download the same data available in the [Lab 2 folder](#) on GitHub and store it in your `data_path` folder. This data was last updated on September 2025. Below, we have included the (commented out) code for downloading the puma occurrences directly from GBIF and also for loading in the file if you have it saved locally in your `data_path` folder.

```
# Take a look at the command "gbif" to see what options exist for importing data.
#?gbif
# Import the occurrence data on Puma concolor - I am using 'nrecs = 300' here
# to cut down on download time. To answer the lab questions, it would be better
# to use all the available records for the species you choose.

# If you want to import the occurrence data directly from GBIF, use this code.
# (Might take a while)
# puma.gbif <- dismo::gbif("Puma", "concolor", geo=T, nrecs=300)

# If you have the occurrence data downloaded from GitHub, use this code to
# load it into R:
puma.gbif <- readr::read_delim(file.path("data/lab2/puma_concolor.csv"),
                              delim = "\t", escape_double = FALSE, trim_ws = TRUE)
```

```
# plot occurrences
par(mfcol=c(1,1))
plot(us$geometry)
points(puma.gbif$decimalLongitude, puma.gbif$decimalLatitude,
       pch=19, cex=0.3, col="blue")
```



```
# important: longitude goes first and hence the order
```

Note: This method of using the “points” function allows you to plot without telling R that the data are spatial in nature (it’s still just a dataframe). You can visualize the data, but you will not be able to do other spatial analyses (e.g., intersections, projections, etc.) until you tell R that this is a spatial object.

```
names(puma.gbif)
```

## [1] "gbifID"	"datasetKey"	"occurrenceID"
## [4] "kingdom"	"phylum"	"class"
## [7] "order"	"family"	"genus"
## [10] "species"	"infraspecificEpithet"	"taxonRank"
## [13] "scientificName"	"verbatimScientificName"	"verbatimScientificNameAu"
## [16] "countryCode"	"locality"	"stateProvince"
## [19] "occurrenceStatus"	"individualCount"	"publishingOrgKey"
## [22] "decimalLatitude"	"decimalLongitude"	"coordinateUncertaintyInM"
## [25] "coordinatePrecision"	"elevation"	"elevationAccuracy"
## [28] "depth"	"depthAccuracy"	"eventDate"
## [31] "day"	"month"	"year"
## [34] "taxonKey"	"speciesKey"	"basisOfRecord"
## [37] "institutionCode"	"collectionCode"	"catalogNumber"
## [40] "recordNumber"	"identifiedBy"	"dateIdentified"
## [43] "license"	"rightsHolder"	"recordedBy"
## [46] "typeStatus"	"establishmentMeans"	"lastInterpreted"
## [49] "mediaType"	"issue"	


```

# remove NA values if they exist in lat or lon
puma1<-subset(puma.gbif, puma.gbif$decimalLatitude != "NA",)
puma1<-subset(puma1, puma1$decimalLongitude != "NA",)
dim(puma1)

```

```
## [1] 19303    50
```

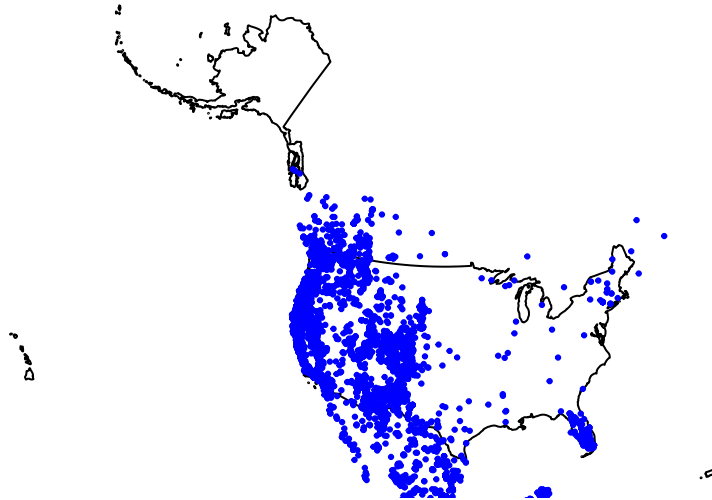
```

# puma.gbif is just a dataframe. We need to tell R that it's a spatial object
# This defines the CRS for the US in WGS 1984 (world geographic 1984).
wgs1984.proj <- CRS("EPSG:4326")

# Reference the columns with "lon" and "lat" to make a spatial points dataframe
puma <- st_as_sf(puma1,
                 coords = c("decimalLongitude", "decimalLatitude"),
                 crs = wgs1984.proj)
# Note: You can also just put "4326" (i.e., the epsg code for WGS84) into the
# "crs =" field and the code will run.
# It's two ways of achieving the same outcome.
# puma <- st_as_sf(puma1,
#                  coords = c("decimalLongitude", "decimalLatitude"),
#                  crs = 4326)

# Note that that WGS84 is a geographic coordinate system. In order to map the
# data, calculate distances, and run spatial operations, we need to project it.
# For this analysis, we will use an Albers equal area projection (i.e., the [US
# National Atlas equal-area projection](https://epsg.io/3899) from before)..
puma.equal <- st_transform(puma, us.atlas.proj)
plot(sf::st_geometry(us.equal))
points(puma.equal, pch=19, cex=0.3, col="blue")

```



```
### CREATE AN EMPTY GRID - this will define the extent of the study area
#What is the dimension of the geometry?
head(us.equal$geometry)
```

```
## Geometry set for 1 feature
## Geometry type: MULTIPOLYGON
## Dimension:      XYZ
## Bounding box:   xmin: -5761986 ymin: -2361733 xmax: 3663387 ymax: 3911055
## z_range:        zmin: 0 zmax: 0
## Projected CRS:  NAD27 / US National Atlas Equal Area
```

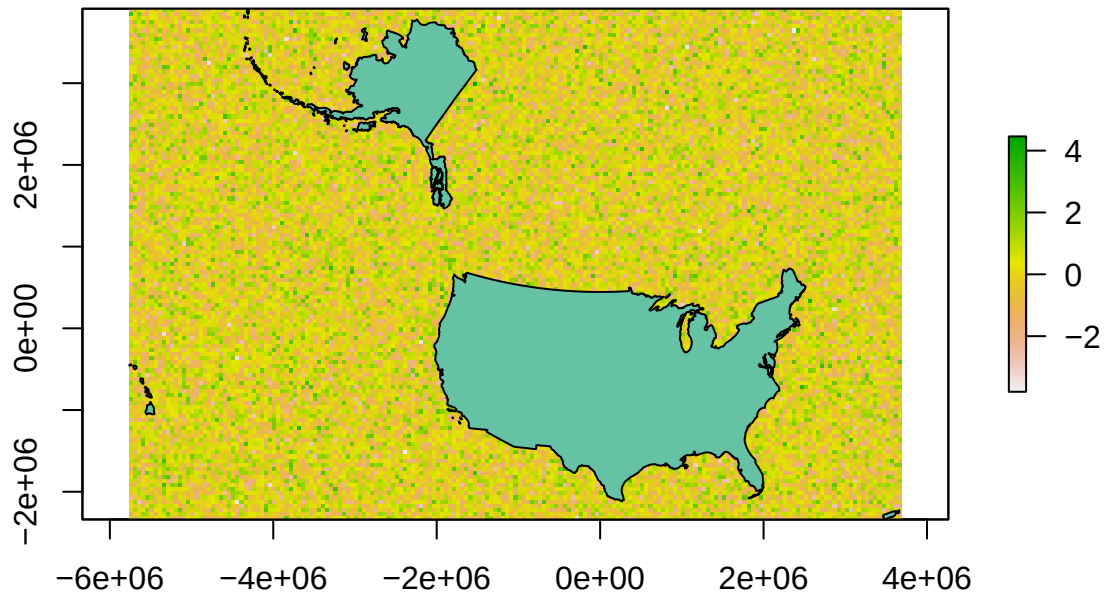
```
#The spatial data contains Z (3D)we need to remove the z dimension before
# creating the raster
# ??st_zm # Pulls up information on the site
us.equal <- sf::st_zm(us.equal)
# units = m
r <- raster::raster(us.equal)
head(us.equal$geometry)
```

```
## Geometry set for 1 feature
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -5761986 ymin: -2361733 xmax: 3663387 ymax: 3911055
## Projected CRS:  NAD27 / US National Atlas Equal Area
```

```

# resolution (grain size in units of us.equal = meters)
res(r) <- 50000
r[] <- rnorm(ncell(r))
plot(r)
plot(us.equal, add=T)

```

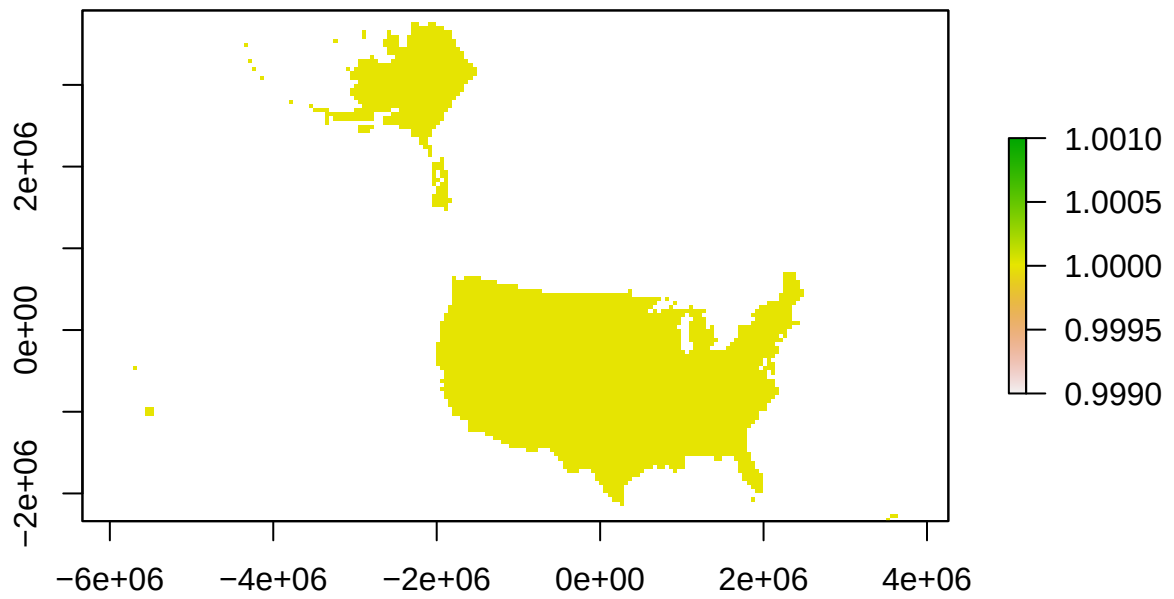


```

r[] <- 0

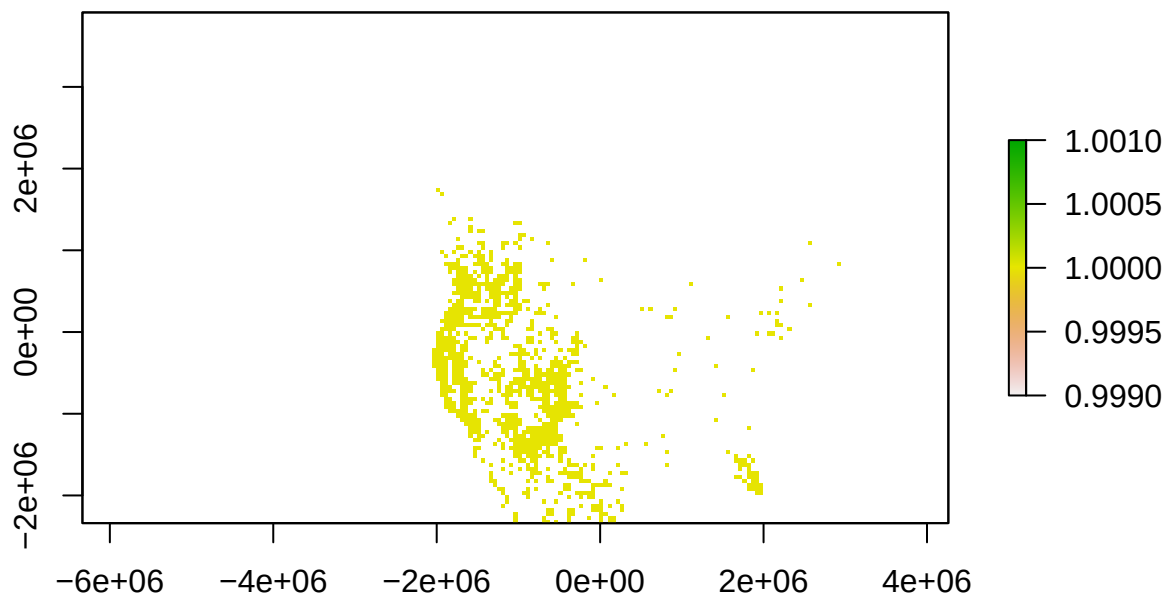
### RASTERIZING (fitting data into a grid)
# rasterizing US boundary
us.raster <- terra::rasterize(us.equal, r, background = NA)
plot(us.raster)

```



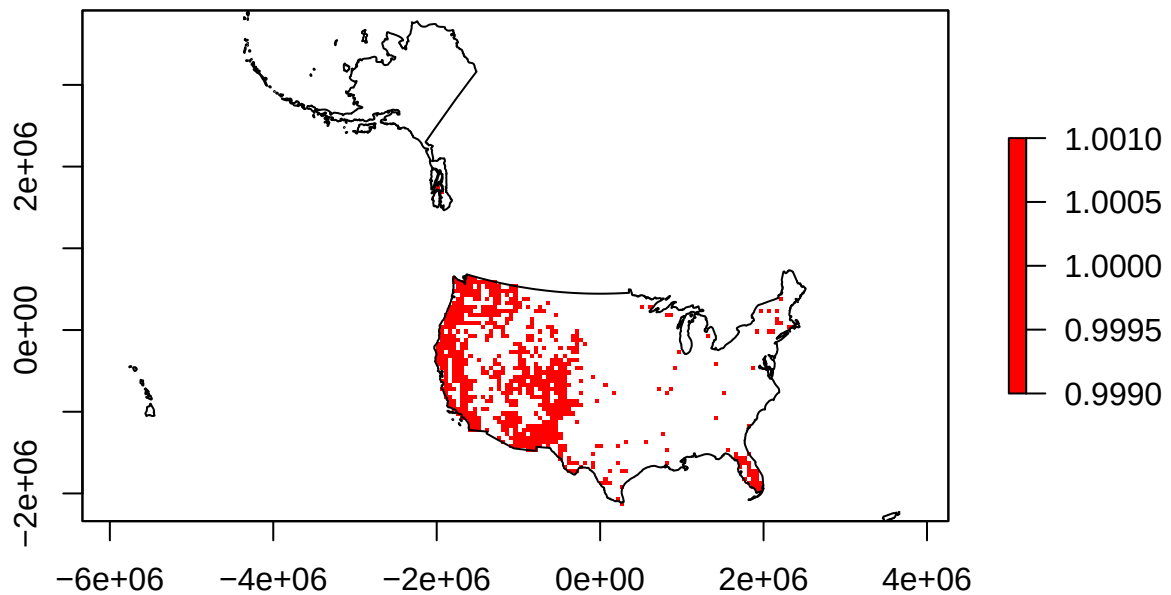
```
# Create column of "1" value to indicate presence and rasterize
puma.equal <- puma.equal %>% dplyr::mutate(presence = 1)
puma.raster <- terra::rasterize(puma.equal, r, field = "presence", background=NA)

plot(puma.raster)
```



```
# limiting the puma point data only to US  
puma.raster <- puma.raster*us.raster
```

```
# add the US outline shapefile  
plot(puma.raster,col='red')  
plot(sf::st_geometry(us.equal),add=T)
```



```

### AGGREGATE TO COARSE GRAIN - puma occurrences
par(mfrow=c(2,2), mai=c(0.1, 0.1, 0.5, 0.1)) # 2x2 plots with margins

# 50 x 50 resolution
plot(puma.raster, axes=FALSE, col='red',
     legend=FALSE, main="50 km x 50 km")
plot(sf::st_geometry(us.equal), add=T)

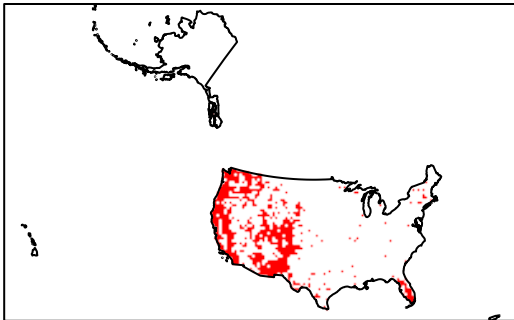
# 100 x 100 resolution
p.coarser <- terra::aggregate(puma.raster, fact=2, fun=max)
plot(p.coarser, axes=FALSE, col='red',
     legend=FALSE, main="100 km x 100 km")
plot(sf::st_geometry(us.equal), add=T)

# 200 x 200 resolution
p.coarser2 <- terra::aggregate(p.coarser, fact=2, fun=max)
plot(p.coarser2, axes=FALSE, col='red',
     legend=FALSE, main="200 km x 200 km")
plot(sf::st_geometry(us.equal), add=T)

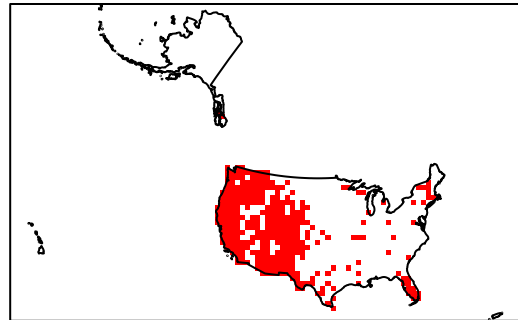
# 400 x 400 resolution
p.coarser4 <- terra::aggregate(p.coarser2, fact=2, fun=max)
plot(p.coarser4, axes=FALSE, col='red',
     legend=FALSE, main="400 km x 400 km")
plot(sf::st_geometry(us.equal), add=T)

```

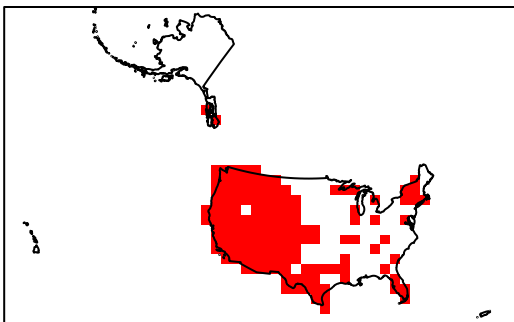
50 km x 50 km



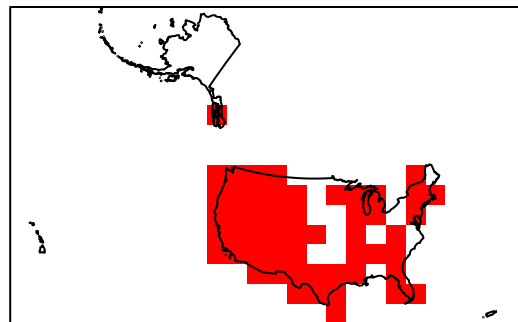
100 km x 100 km



200 km x 200 km



400 km x 400 km



```
### Save your Workspace ###  
## If you want to save your workspace and all the R objects within it:  
#save.image(file.path(output_path, "lab2.RData"))
```

ASSIGNMENT - Hand in a R Markdown-produced PDF to D2L including the following:

1. Repeat the process above, but create 2 side-by-side plots for 2 new resolutions of your choice. Create both of the plots at the extent of the US Northeast (or if you use another species and it's not in the Northeast, clip it to another US region). Clip the rasters so they only plot for the extent of the Northeast (or another region depending on the species you choose).
- a) ADD A SCALEBAR: See these potential approaches: scalebar.R: <https://rdr.io/cran/raster/src/R/scalebar.R> and <https://rdr.io/cran/raster/man/scalebar.html>. Or: <https://www.rdocumentation.org/packages/raster/versions/2.6-7/topics/scalebar> hint: look at the units (and extent) of a raster by typing the name of the raster at the R prompt.
- b) ADD A NORTH ARROW: hint: <https://www.rdocumentation.org/packages/GISTools/versions/0.7-4/topics/North%20Arrow> - this relies on the R package, GISTools.
- c) CHANGE COLORS: Make each one of the plots differ in color for pixels of the species occurrence raster.
- d) OVERLAY the species occurrence points on one of your plotted maps. Use a contrasting color for the points.

- e) ADD A LEGEND. hint: <https://biologyforfun.wordpress.com/2013/03/11/taking-control-of-the-legend-in-raster-in-r/> Some tips on plotting LEGENDS with rasters (“distribution” below) and shapefiles (“occurrence” below) together. “left” tells R where to put the legend (lower left). You can also specify actual coordinates. pch is the base R code for the point shape: <http://www.statmethods.net/advgraphs/parameters.html> - here it’s referencing 21 and 15 which are the circles and squares for “occurrence” and “distribution” respectively. Assign their colors with the “col=” command, and their size with “pt.cex”. Adding a title to the legend is important, otherwise we don’t know what we’re looking at. ?legend will tell you more.
2. Repeat 1a-e. above, but this time reproject the data into another projection for North America (your choice).
 3. Briefly answer the following: How do the different projections and resolutions change your interpretation of the distribution of the species? Keep in mind that distribution does not just include extent or range size. What would be an appropriate resolution and extent of occurrence data for the species you chose if you are trying to represent its full distribution? Which type of Coordinate System (projected or geographic), if any, would be more appropriate to use with these data if you wanted to measure distances between a) two extreme ends of the species range? and b) two nearby species occurrences locations?

Here are some useful bits of code:

```
# Regions of US shapefile
download.file("http://www2.census.gov/geo/tiger/GENZ2014/shp/cb_2014_us_region_20m.zip",
              dest=file.path(data_path,"usr.zip"), mode="wb")
unzip( file.path(data_path,"usr.zip"), exdir = data_path)

# read in the shapefile
usr <- sf::read_sf(file.path(data_path,"cb_2014_us_region_20m.shp"))
## take a look at the shapefile info
summary(usr)
## follow the steps above for projecting it to WGS 1984, then to US National
# Atlas Equal-Area.
## Hint for plotting region: extract a portion of a shapefile and create a new
# one.
## Look at the first 6 lines of data in the shapefile attribute table
# (you need to create usr.equal first).
# head(usr.equal)
## make a shapefile of just Northeast
# ne <- usr.equal[usr.equal$NAME == "Northeast",]
```

Homework in preparation for Lab 3:

In Lab 3 we will be working with landcover data from the National Land Cover Database (NLCD). Read about NLCD here (<https://www.mrlc.gov/>) and view the short video: <https://www.mrlc.gov/nlcd-2016-video>.

OPTIONAL MSU HPCC info: In past years we have used the MSU HPCC for this lab, but we will not be doing so this year. We will be using a smaller chunk of the data so you can run it on your computer.

If you’re interested in learning about the HPCC here is the go-to Introduction website: <https://icer.msu.edu/for-hpcc-users/Topic-of-the-Month/Introduction-to-HPCC-Training-Site>

There are also Intro to HPCC workshops offered monthly through iCER; free but registration required <https://icer.msu.edu/upcoming-workshops>

If you are not familiar with basic Linux commands, check out the HPCC Intro to Linux online courses at the wiki link above. You can also check them out here: <https://linuxconfig.org/bash-scripting-tutorial-for-beginners#h1-bash-shell-scripting-definition>.

This work is licensed under a Licensed under CC-BY 4.0 2025; Website created by Phoebe Zarnetske.